

Deep Learning Ultimate Cheat Sheet

PyTorch, Architectures & Training

1 PyTorch Basics

Tensors (The Core)

```
import torch

# Creation
x = torch.tensor([1, 2, 3])
r = torch.rand(2, 3) # Random [0,1)
z = torch.zeros(2, 2)
o = torch.ones(2, 2)

# Dimensions
s = x.shape # or x.size()
```

Device (GPU/MPS)

```
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)
```

Autograd (Gradients)

```
# Track gradients
w = torch.tensor(1.0, requires_grad=True)
y = w * 2
z = y ** 2

# Backpropagation
z.backward()
print(w.grad) # dz/dw
```

2 Neural Networks (nn.Module)

Defining a Network

```
import torch.nn as nn

class MyNet(nn.Module):
    def __init__(self):
        super().__init__()
        # Define Layers
        self.fc1 = nn.Linear(10, 50)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(50, 1)

    def forward(self, x):
        # Define Flow
        x = self.fc1(x)
        x = self.relu(x)
        return self.fc2(x)

model = MyNet().to(device)
```



MLP Architecture

Common Layers

```
# Fully Connected
l = nn.Linear(in_features, out_features)

# Convolutional (Vision)
c = nn.Conv2d(in_channels, out_channels,
              kernel_size=3, padding=1)

# Recurrent (Sequence)
r = nn.LSTM(input_size, hidden_size)

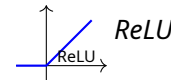
# Dropout (Regularization)
d = nn.Dropout(p=0.5)

# Normalization
bn = nn.BatchNorm2d(num_features)
```

3 Activation Functions

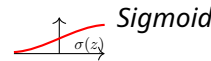
ReLU (Rectified Linear Unit)

```
# Use in Hidden Layers
act = nn.ReLU()
# f(x) = max(0, x)
```



Sigmoid

```
# Use in Output (Binary Class)
act = nn.Sigmoid()
# Range: (0, 1)
```



Softmax

```
# Use in Output (Multi-Class)
# Sums to 1 across dim
act = nn.Softmax(dim=1)
```

4 Loss & Optimization

Loss Functions

```
# Regression
criterion = nn.MSELoss()

# Binary Classification
criterion = nn.BCELoss()

# Multi-Class Classification
# (Takes raw logits, no softmax needed)
criterion = nn.CrossEntropyLoss()
```

Optimizers

```
import torch.optim as optim

# Stochastic Gradient Descent
opt = optim.SGD(model.parameters(), lr=0.01)

# Adam (Adaptive)
opt = optim.Adam(model.parameters(), lr=1e-3)
```

5 The Training Loop

```
epochs = 10
for epoch in range(epochs):
    # Training Mode
    model.train()

    for x_batch, y_batch in dataloader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        # 1. Forward Pass
        preds = model(x_batch)
        loss = criterion(preds, y_batch)

        # 2. Backward Pass
        optimizer.zero_grad() # Clear gradients
        loss.backward()        # Calc gradients
        optimizer.step()        # Update weights

    print(f"Epoch {epoch}, Loss: {loss.item()}")
```

6 Data Loading

Dataset Class

```
from torch.utils.data import Dataset, DataLoader

class MyData(Dataset):
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        return self.data[idx], self.label[idx]
```

DataLoader

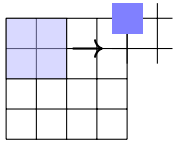
```
# Batches, shuffling, parallel loading
loader = DataLoader(dataset, batch_size=32,
                    shuffle=True)
```

7 Computer Vision (CNNs)

Operations

- **Convolution:** Feature extraction.
- **Pooling:** Downsampling (Max/Avg).
- **Flatten:** Prepare for Linear layer.

```
nn.Sequential(
  nn.Conv2d(3, 16, 3), # In: 3 (RGB)
  nn.ReLU(),
  nn.MaxPool2d(2, 2), # Downsample
  nn.Flatten(),
  nn.Linear(16 * 13 * 13, 10)
)
```



Convolution

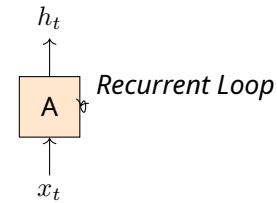
8 Sequences (RNNs)

RNN / LSTM / GRU

```
# Input: (Batch, Seq_Len, Features)
rnn = nn.LSTM(input_size=10, hidden_size=20,
              batch_first=True)
```

```
# Output: (output, (h_n, c_n))
```

```
out, (h, c) = rnn(x)
```



Recurrent Loop

Transformers (Modern)

```
layer = nn.TransformerEncoderLayer(
  d_model=512, nhead=8
)
transformer = nn.TransformerEncoder(layer, 6)
```

9 Saving Models

```
# Save Weights (Recommended)
torch.save(model.state_dict(), "model.pth")
```

```
# Load Weights
model = MyNet()
model.load_state_dict(torch.load("model.pth"))
model.eval() # Set to evaluation mode
```

10 Common Pitfalls

- **Shape Mismatch:** Check `x.shape` inside `forward()`.
- **Device Error:** Ensure both model and data are on `cuda` or `cpu`.
- **Forgot `zero_grad()`:** Gradients accumulate by default.
- **Training vs Eval:** Use `model.train()` and `model.eval()` for Dropout/BatchNorm.